



EDITORA AGÊNTICA

1ª Edição · 2026

AUTORIZAÇÃO DE POUSO: RELEASE E OBSERVABILIDADE

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ENGENHEIRO DE SOFTWARE & MAKER

Heverton Eduardo Peres

Autorização de Pouso: Release e Observabilidade

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P885a PERES, Heverton Eduardo

Autorização de Pouso: Release e Observabilidade / Heverton Eduardo
Peres. – São Paulo : Fábrica Agêntica de Livros,
2026.

30 p. ; 21 cm.

ISBN 978-65-00000-10-8

1. Autorização. 2. Pouso. 3. Release. 4. Observabilidade. I. Título.

CDD 004.6

Sumário

Autorização de Pouso: Release e Observabilidade	6
Capítulo 7: Autorização de Pouso: Release e Observabilidade	7
Introdução	7
Explica	7
Ilustra	8
Técnica	9
Release Reproduzível com Build Imutável	9
Deploy Canário com Gate de Saúde	9
Observabilidade do Agente: a Caixa-Preta do Comportamento	10
O Modelo de Escalonamento de Incidente de Release	11
O Fallback de Ambiente: Cuspir Comandos Prontos	11
O Pipeline de Release com Estágios Declarados	12
O Modelo de Decisão de Rollback em Tempo Real	12
O Canário como Contrato de Confiança	13
O Canário como Contrato de Confiança	14
A Caixa-Preta do Agente em Produção	14
O Modelo de Sinais Vitais do Release	14
O Modelo de Estimativa de Páginas por Caracteres	14
O Modelo de SLO como Contrato de Produção	15
O Modelo de Rollback Automático	16
O Registro de Estágios do Deploy	17
O Modelo de Análise Pós-Mortem de Release	17
O Contrato de Observabilidade do Release	18
O Modelo de Verificação de Release com Checklist	18
O Runbook de Incidente como Artefato de Aterrissagem	19
O Runbook de Incidente como Artefato de Aterrissagem	20
A Redundância do Plano de Pouso	20
O Fallback como Cidadão de Primeira Classe	20
O Diário do Release	21
Passos para Autorizar o Pouso	21
Aplica	21
Conclusão	22
Por que este capítulo importa	22
Conceitos-chave deste capítulo	23

Checklist para aplicar hoje	23
Perguntas que você deve se fazer	23
Glossário rápido	24
O erro mais comum nesta fase	24
Um exemplo concreto para fixar	24
A rotina de quem já opera assim	25
O que não fazer: os anti-padrões mais comuns	25
Como medir o progresso na prática	25
O papel do líder neste capítulo	26
Perguntas frequentes honestas	26
Um convite para a prática deliberada	26
Síntese para levar com você	27
De onde veio a necessidade desta mudança	27
Conversando com quem resiste	27
O dia a dia no detalhe	28
O custo invisível que decide tudo	28
Uma visão de longo prazo	29
Próximos Passos	29

Autorização de Pouso: Release e Observabilidade

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 7: Autorização de Pouso: Release e Observabilidade

Introdução

No Capítulo 6, você acionou o radar e aprendeu a verificação adversarial em três camadas — máquina, revisor independente e humano — com evidência antes de afirmação. O voo agora está na final: chegou a hora da autorização de pouso. Este capítulo cobre a Fase 6 (entregar) e a Fase 7 (operar) do SDLC AI-first: release reproduzível, deploy gradual e observabilidade do comportamento do próprio agente.

Você vai aprender por que “release = artefato reproduzível” é uma regra de engenharia (não de processo), como desenhar deploy canário com rollback, e como monitorar não apenas o software que você entregou — mas o comportamento do agente que o produziu, no ambiente de produção.

Explica

No SDLC clássico, entregar é quase um ato de fé: o build roda na máquina de um desenvolvedor, o deploy é um script meio-documentado, e a operação descobre os problemas quando o cliente liga. O AI-first não elimina a complexidade — mas a torna **rastreável e reversível**.

A primeira regra é a reprodutibilidade. Um release é um artefato reproduzível quando o mesmo commit, no mesmo ambiente, produz o mesmo binário — sempre. Isso parece óbvio, mas o número de organizações que “compila na minha máquina” é maior do que a indústria admite. A reprodutibilidade é o pré-requisito de tudo o que vem depois: se você não consegue reconstruir o artefato, não consegue nem diagnosticar o incidente.

A segunda regra é o deploy gradual. Em vez de substituir tudo de uma vez (big bang), o release caminha por estágios: canário (uma fração de usuários), grupos progressivos, e só então a totalidade. Cada estágio é uma oportunidade de observar e reverter. A literatura de DevOps

(DORA) documenta que a capacidade de reverter rapidamente é um dos maiores preditores de estabilidade organizacional.

A terceira regra é a observabilidade. Monitorar não é medir uptime — é responder à pergunta “o que está acontecendo agora, e por quê?”. Logs, métricas e rastreios distribuídos formam a caixa-preta do sistema: quando algo falha, a caixa-preta conta a história completa. No contexto AI-first, a caixa-preta precisa incluir também o **comportamento do agente**: qual decisão ele tomou, com base em qual contexto, produzindo qual artefato.

Por que observar o agente é diferente de observar o software? Porque o agente é um componente novo no sistema — um produtor de mudanças com comportamento probabilístico. Um deploy de código humano muda de forma previsível; um deploy de mudanças agênticas pode variar em qualidade, escopo e até direção. A observabilidade do agente é o que transforma esse comportamento variável em insumo de decisão.

A dimensão do fallback completa o quadro. Ambientes bloqueiam, dependências falham, provedores mudam contratos. A regra operacional do SDLC AI-first: quando o ambiente bloqueia a automação, o processo deve **cuspir os comandos prontos** para execução manual — nunca parar em silêncio. A entrega nunca fica refém de uma única ferramenta.

E há a lição do relatório DORA de 2025 sobre IA: a velocidade de entrega sobe com a adoção de IA, mas a estabilidade exige que a capacidade de observação e reversão suba junto. Quem entrega mais rápido sem observar mais cedo está apostando que o radar continua funcionando — sem evidência.

Ilustra

Uma aterrissagem comercial raramente é um movimento único. O piloto se aproxima, a torre autoriza, o avião toca a pista, desacelera e só então libera a pista para o próximo. Em condições adversas, o piloto arremete — aborta o pouso e volta para uma nova tentativa. Arremeter não é falha; é o plano B funcionando.

O deploy gradual é essa aterrissagem: toque a pista com uma fração (canário), confirme que está firme, e só então libere o tráfego inteiro. O rollback é a arremetida: se o toque foi ruim, sobe de novo e tenta outra abordagem — sem vergonha e sem culpa.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você grava o padrão: cada estágio tem sua própria autorização de pouso — e a anomalia em qualquer estágio aciona a arremetida, não o desespero.

Técnica

Release Reprodutível com Build Imutável

A reprodutibilidade começa no build. O exemplo em Docker mostra um build imutável: o mesmo commit produz a mesma imagem, com dependências fixadas.

```
# Build reprodutível: dependências fixadas e etapa de build isolada
FROM node:20-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --frozen-lockfile
COPY src ./src
COPY tsconfig.json ./
RUN npm run build

FROM node:20-slim AS runtime
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

O `npm ci --frozen-lockfile` é a garantia: as dependências exatas do lockfile, não “as mais recentes compatíveis”. O build é reprodutível porque as entradas são o commit e o lockfile — nada mais.

Deploy Canário com Gate de Saúde

O gate de saúde é a autorização de pouso automatizada. O script abaixo simula o controle: só avança para o próximo estágio se as métricas do estágio atual estiverem dentro do limiar.

```
import json
import time
from dataclasses import dataclass

@dataclass
class Metrica:
    nome: str
    valor: float
    limite: float
```

```
def saudavel(self) -> bool:
    return self.valor <= self.limite

def obter_metricas_canario() -> list:
    # Simulacao: em producao, vem do sistema de observabilidade
    return [
        Metrica("taxa_erro", 0.4, limite=1.0),
        Metrica("latencia_p95_ms", 320, limite=500),
        Metrica("deploy_fracassado", 0.0, limite=0.0),
    ]

def gate_de_saude(estagio: str) -> None:
    metricas = obter_metricas_canario()
    ruins = [m.nome for m in metricas if not m.saudavel()]
    if ruins:
        print(f"[ARREITAR] estagio={estagio} metricas ruins: {'',
        '.join(ruins)}")
        print("[ROLLBACK] revertendo para versao anterior")
        return
    print(f"[AUTORIZADO] estagio={estagio} avanca para o proximo grupo")

if __name__ == "__main__":
    gate_de_saude("canario_2pct")
    time.sleep(0.1)
```

A moral: a decisão de avançar ou arremeter é automatizada e baseada em métrica — não em “senti que estava ok”.

Observabilidade do Agente: a Caixa-Preta do Comportamento

Além do software, o SDLC AI-first observa o produtor. O schema abaixo captura a decisão do agente em cada fase:

```
{
  "evento": "agente_decidiu",
  "timestamp": "2026-08-02T14:31:00Z",
  "fase": "build",
  "agente": "redator-pagamentos",
  "decisao": "implementar interface ProvedorPagamento",
  "contexto_consumido": {"tokens_entrada": 18400, "tokens_saida": 3200},
  "artefato": "src/pagamentos/interface.ts",
  "evidencia": "teste login_fluxo_feliz verde em 1.2s",
```

```
"revisao": {"camada": "adversarial", "resultado": "nao_refutado"}
}
```

Esse log de evento vira um painel que responde perguntas que o uptime não responde: onde o agente gasta tokens? Qual decisão produziu retrabalho? Qual contexto levou a qual artefato? É o radar da torre apontado para o próprio piloto.

O Modelo de Escalonamento de Incidente de Release

O incidente em produção não avisa — o escalonamento precisa ser automático. O modelo abaixo decide quem acionar conforme a severidade do incidente de release:

```
class EscalonamentoDeIncidente:
    def __init__(self):
        self.fila = []

    def escalar(self, incidente, severidade):
        acionados = {'baixa': ['time_de_plantao'], 'media':
['time_de_plantao', 'sre'],
                    'alta': ['time_de_plantao', 'sre', 'comandante'],
                    'critica': ['time_de_plantao', 'sre', 'comandante',
'executivo']}
        self.fila.append({'incidente': incidente, 'severidade': severidade,
'acionados': acionados[severidade]})
        return self.fila[-1]

e = EscalonamentoDeIncidente()
print(e.escalar('pagamentos_500', 'critica'))
```

A tabela de escalonamento elimina a indecisão do pânico: severidade crítica aciona até o executivo, sem reunião para decidir quem chamar. O padrão também se acumula — se a fila mostra três incidentes críticos na mesma semana, o problema não é operacional, é de qualidade de release, e a conversa muda de nível. Escalonamento automático não substitui julgamento; remove a paralisia entre o incidente e a ação.

O Fallback de Ambiente: Cuspir Comandos Prontos

Quando a automação é bloqueada pelo ambiente, o processo entrega os comandos prontos. O exemplo em PowerShell mostra o padrão — nunca parar em silêncio:

```
# Fallback de compilacao quando o sandbox bloqueia a automacao
$slug = "sdlc-ai-first"
Write-Host "Sandbox bloqueou a execucao automatica."
```

```
Write-Host "Execute manualmente no terminal local:"
Write-Host "  cd fabrica-de-livros"
Write-Host "  python compilar-para-pdf.py $slug --paginas-exatas"
Write-Host "  powershell -ExecutionPolicy Bypass -File scripts/converter-
md-pdf.ps1 -Slug $slug"
```

O padrão é universal: a esteira nunca morre em silêncio — degrada com instruções executáveis.

O Pipeline de Release com Estágios Declarados

O pipeline de release AI-first declara os estágios como dado — não como passo de um script ad hoc. O YAML abaixo é o modelo que a esteira interpreta para orquestrar o deploy gradual.

```
release:
  artefato: "imagem: sha256:<hash>"
  estagios:
    - nome: canario
      percentual: 2
      gate: metricas_saude
      duracao_observacao_min: 30
    - nome: grupo_progressivo
      percentual: 25
      gate: metricas_saude
      duracao_observacao_min: 60
    - nome: producao_total
      percentual: 100
      gate: metricas_saude
  rollback:
    gatilho: "taxa_erro > 1.0 || latencia_p95 > 500"
    acao: "reverter para imagem anterior e notificar incidente"
  evidencia_obrigatoria:
    - "saida_canario.txt"
    - "saida_grupo.txt"
    - "aprovacao_humana.txt"
```

Cada estágio declara percentual, gate e duração de observação. A esteira executa, coleta evidência em arquivo e só avança quando o gate passa — a autorização de pouso automatizada, com o humano como última instância.

O Modelo de Decisão de Rollback em Tempo Real

O rollback não é uma reação — é uma decisão que precisa ser tomada em segundos, com dados. O modelo abaixo consolida os sinais vitais do release (erros, latência, taxa de sucesso) e devolve a recomendação de pouso, desvio ou volta:

```
def decidir_rollback(sinais):
    criticos = 0
    for nome, limite, atual in sinais:
        if atual > limite:
            criticos += 1
    if criticos >= 2:
        return {'acao': 'rollback_imediato', 'sinais_criticos': criticos}
    if criticos == 1:
        return {'acao': 'desvio_para_standby', 'sinais_criticos': criticos}
    return {'acao': 'manter_release', 'sinais_criticos': criticos}

sinais = [('erros_500', 5, 12), ('latencia_p95', 800, 1400),
          ('taxa_sucesso', 0.98, 0.93)]
print(decidir_rollback(sinais))
```

Dois sinais críticos simultâneos significam problema sistêmico — esperar para ver só piora. Um sinal crítico isolado pode ser resíduo de tráfego, e o desvio para o cluster standby dá tempo de investigar sem expor usuários. A regra é explícita para que o agente não precise interpretar: a esteira decide por tabela, e o humano só revisa o que foge à tabela.

O Canário como Contrato de Confiança

A caixa-preta do agente precisa de um painel — senão os eventos registrados morrem em arquivos que ninguém consulta. O painel mínimo responde a cinco perguntas:

Pergunta	Métrica	Origem do dado
Onde o agente gasta tokens?	tokens por fase	log de sessão
Qual decisão produziu retrabalho?	refutações por decisão	pareceres da Fase 5
Qual contexto levou a qual artefato?	mapeamento contexto → artefato	evento de decisão
Quanto tempo cada fase consumiu?	duração por fase	timestamps
Qual agente tem pior taxa de sucesso?	sucesso por agente	resultado do build

O painel transforma a observabilidade de repositório de dados em instrumento de decisão: o Comandante consulta o painel, não a intuição, para calibrar a próxima iteração.

O Canário como Contrato de Confiança

O canário não é apenas uma técnica de deploy — é um contrato de confiança entre o time e a produção. Quando 2% dos usuários recebem a mudança e as métricas seguem saudáveis, o restante do tráfego é autorizado com base em evidência, não em esperança. Essa transição de confiança baseada em dados é o mesmo princípio do radar do Capítulo 6: cada estágio produz evidência, e a evidência autoriza o próximo estágio. Sem canário, o deploy é um salto no escuro com os olhos vendados.

A Caixa-Preta do Agente em Produção

A observabilidade do agente não termina no deploy. Em produção, o agente pode continuar operando — automatizando correções, gerando relatórios, sugerindo mudanças. A caixa-preta precisa registrar cada decisão com o mesmo rigor da Fase 5: o que foi decidido, com base em qual contexto, produzindo qual artefato, com qual evidência. Quando um incidente ocorre, a caixa-preta é a primeira fonte de verdade — e a única que conta a história completa do produtor.

O Modelo de Sinais Vitais do Release

O contrato de observabilidade precisa de sinais vitais — o conjunto mínimo de métricas que define a saúde do release. O modelo abaixo é o conjunto canônico:

Sinal vital	Pergunta que responde	Limiar crítico
Taxa de erro	O serviço está falhando?	> 1% em 10 min
Latência p95	O serviço está lento?	> 500 ms em 15 min
Throughput	O tráfego chegou?	queda > 50%
Fila	O processamento está acumulando?	> 1000 itens
Disponibilidade	O serviço está de pé?	< 99,9%

Os sinais vitais são o painel da cabine do release: cada um tem limiar e ação associada — e o rollback automático do capítulo aciona quando o sinal cruza o limiar. O Comandante não monitora tudo; monitora o mínimo que define a vida.

O Modelo de Estimativa de Páginas por Caracteres

A estimativa de tamanho da obra — o requisito que aparece em toda auditoria editorial — usa a conversão ABNT de aproximadamente 2.500 caracteres por página. O modelo abaixo ajuda o Comandante a dimensionar releases e documentação:

```
def estimar_paginas(caracteres: int, por_pagina: int = 2500) -> int:
    return max(1, round(caracteres / por_pagina))

def estimar_caracteres(paginas: int, por_pagina: int = 2500) -> int:
    return paginas * por_pagina

print(f"10.000 chars ~ {estimar_paginas(10_000)} paginas")
print(f"150 paginas ~ {estimar_caracteres(150):,} chars".replace(",", "."))
```

A estimativa por caracteres é o instrumento de planejamento: cada capítulo, cada release e cada documento têm um alvo de tamanho — e a auditoria verifica o alvo com a mesma régua.

O Modelo de SLO como Contrato de Produção

Produção precisa de números acordados antes do release. O modelo abaixo valida se o release atende aos SLOs (Service Level Objectives) configurados e acende o alerta quando um deles está em risco:

```
class ValidadorDeSLO:
    def __init__(self, slos):
        self.slos = slos

    def avaliar(self, metricas):
        resultado = []
        for nome, limite in self.slos.items():
            atual = metricas.get(nome)
            if atual is None:
                resultado.append({'slo': nome, 'status': 'sem_dados'})
            elif atual >= limite:
                resultado.append({'slo': nome, 'status': 'ok', 'atual':
atual, 'limite': limite})
            else:
                resultado.append({'slo': nome, 'status': 'em_risco',
'atual': atual, 'limite': limite})
        return resultado

slos = {'disponibilidade': 0.995, 'sucesso_pagamentos': 0.99}
metricas = {'disponibilidade': 0.992, 'sucesso_pagamentos': 0.994}
print(ValidadorDeSLO(slos).avaliar(metricas))
```

O SLO é o contrato entre a esteira e a operação: a disponibilidade de 99.5% é prometida antes do release, não descoberta depois. Quando a métrica fica em risco, o painel mostra a tendência

e o tempo projetado até violar o contrato — dando ao comandante a janela para decidir antes do incidente. Sem SLO, o “funciona em produção” é opinião; com SLO, é número.

O Modelo de Rollback Automático

O rollback não é um comando — é um sistema. O modelo de rollback automático decide quando reverter com base nas métricas do contrato de observabilidade, sem esperar um humano:

```
from dataclasses import dataclass

@dataclass
class GatilhoRollback:
    metrica: str
    limiar: float
    janela_min: int

    def disparou(self, valor: float) -> bool:
        return valor > self.limiar

GATILHOS = [
    GatilhoRollback("taxa_erro", limiar=1.0, janela_min=10),
    GatilhoRollback("latencia_p95_ms", limiar=500, janela_min=15),
]

def monitorar(metricas: dict) -> None:
    for gatilho in GATILHOS:
        valor = metricas.get(gatilho.metrica, 0.0)
        if gatilho.disparou(valor):
            print(f"[ROLLBACK] {gatilho.metrica} {valor} > {gatilho.limiar}")
            return
    print("[SAUDAVEL] nenhum gatilho disparado; promocao autorizada")

monitorar({"taxa_erro": 0.4, "latencia_p95_ms": 320})
monitorar({"taxa_erro": 2.1, "latencia_p95_ms": 320})
```

O rollback automático é a arremetida programada: o sistema reverte sozinho quando o limiar é violado — e o humano é informado depois, com a evidência. A decisão de reverter não espera reunião; espera métrica.

O Registro de Estágios do Deploy

O deploy gradual precisa de registro — cada estágio com timestamp, percentual e resultado. O registro abaixo é a trilha de aterrissagem:

```
{
  "deploy": "pagamentos-v2.4",
  "estagios_executados": [
    {"estagio": "canario", "percentual": 2, "inicio": "2026-08-02T14:00Z",
      "fim": "2026-08-02T14:30Z", "resultado": "saudavel"},
    {"estagio": "grupo", "percentual": 25, "inicio": "2026-08-02T14:31Z",
      "fim": "2026-08-02T15:31Z", "resultado": "saudavel"},
    {"estagio": "total", "percentual": 100, "inicio": "2026-08-02T15:32Z",
      "fim": "2026-08-02T15:35Z", "resultado": "saudavel"}
  ],
  "rollback_acionado": false
}
```

O registro de estágios é a caixa-preta da entrega: se o deploy falhar, a trilha mostra exatamente onde e quando — e o debriefing (Capítulo 8) parte de dados, não de memória.

O Modelo de Análise Pós-Mortem de Release

Todo release termina — bom ou ruim — com análise. O modelo abaixo estrutura o pós-mortem do release, capturando o que foi observado, o que quebrou e a lição em formato padronizado:

```
class PosMortemDeRelease:
    def __init__(self, release):
        self.release = release
        self.observacoes = []

    def registrar_observacao(self, momento, sinal, valor, limite):
        self.observacoes.append({'momento': momento, 'sinal': sinal,
            'valor': valor, 'limite': limite})

    def gerar_relatorio(self):
        desvios = [o for o in self.observacoes if o['valor'] > o['limite']]
        return {'release': self.release, 'sinais_observados':
            len(self.observacoes),
                'desvios': len(desvios), 'desvio_detalhes': desvios,
                'veredito': 'saudavel' if not desvios else 'com_ressalvas'}

pm = PosMortemDeRelease('release-45')
pm.registrar_observacao('pos_deploy_5min', 'latencia_p95', 750, 800)
pm.registrar_observacao('pos_deploy_30min', 'erros_500', 18, 5)
print(pm.gerar_relatorio())
```

O pós-mortem estruturado transforma cada release em dado de aprendizado: o desvio de erros 500 após 30 minutos, invisível na checagem imediata, aparece no relatório e vira candidato a incidente. Sem o formato padronizado, o pós-mortem vira reunião de memória — com ele, vira registro consultável que alimenta o banco de lições da parte IV. O ciclo do release só está completo quando o relatório do pós-mortem é arquivado.

O Contrato de Observabilidade do Release

Todo release deveria nascer com um contrato de observabilidade — a lista do que será monitorado, com limiares e ações. O contrato abaixo é o modelo que acompanha o artefato na jornada para produção:

```
contrato_observabilidade:
  release: "pagamentos-v2.4"
  metricas_criticas:
    - nome: taxa_erro
      limiar: 1.0
      janela_min: 10
      acao: alerta + rollback automatico
    - nome: latencia_p95
      limiar: 500
      janela_min: 15
      acao: alerta + investigacao
    - nome: fila_pagamentos
      limiar: 1000
      janela_min: 5
      acao: alerta critico + oncall
  logs_obrigatorios:
    - evento_agente_decidiu
    - evento_artefato_produzido
    - evento_merge_autorizado
  sre_objetivos:
    - "99.9% de disponibilidade no primeiro mes"
    - "rollback < 10 minutos em qualquer estagio"
```

O contrato responde a pergunta que mata releases ingênuos: “o que significa saudável, e o que fazemos quando não é?”. Sem contrato, o monitoramento é um painel bonito sem decisão associada.

O Modelo de Verificação de Release com Checklist

O release só decola se o checklist de pré-flight está completo. O modelo abaixo verifica as condições de decolagem — backup, teste canário, rollback testado, runbook atualizado — e bloqueia o release no que faltar:

```
PRÉ_FLIGHT = [
    {'item': 'backup_banco', 'descricao': 'backup do banco verificado'},
    {'item': 'canario_verde', 'descricao': 'canario passou nos testes'},
    {'item': 'rollback_treinado', 'descricao': 'rollback executado em staging'},
    {'item': 'runbook_atualizado', 'descricao': 'runbook de incidente atualizado'},
]

def verificar_pre_flight(condicoes):
    faltantes = [p['item'] for p in PRÉ_FLIGHT if not condicoes.get(p['item'])]
    return {'decolagem_liberada': not faltantes, 'faltantes': faltantes}

print(verificar_pre_flight({'backup_banco': True, 'canario_verde': True, 'rollback_treinado': False, 'runbook_atualizado': True}))
```

O checklist converte a cultura de release em procedimento: o rollback treinado em staging não é detalhe — é condição de decolagem. A esteira não autoriza o release que não cumpriu o pré-flight, e o agente não pode contornar com autoridade porque a autoridade de release também está no contrato. O checklist é deliberadamente pequeno: quatro itens fáceis de verificar, fáceis de lembrar e difíceis de esquecer no meio da pressão.

O Runbook de Incidente como Artefato de Aterrissagem

Vamos juntar a teoria do capítulo em um exemplo completo. A feature é o faturamento recorrente — e o release segue o modelo de cinco etapas:

1. **Build imutável:** o mesmo commit gera a mesma imagem (`npm ci --frozen-lockfile` + Docker multi-stage), com hash registrado.
2. **Canário 2%:** 2% dos clientes recebem a mudança; o gate de saúde observa taxa de erro e latência por 30 minutos.
3. **Grupo 25%:** expande para 25%, com observação de 60 minutos.
4. **Produção total:** 100%, com alerta automático se o contrato de observabilidade for violado.
5. **Rollback ensaiado:** o comando de reversão é testado em staging na semana anterior — a arremetida pronta.

O roteiro abaixo simula a promoção de estágio com gate de saúde:

```
import time

def promover_estagio(nome: str, percentual: int, saudavel: bool) -> None:
    if not saudavel:
        print(f"[ARRETER] estagio={nome} abortado; rollback acionado")
```

```

    return
    print(f"[PROMOVER] estagio={nome} percentual={percentual}% autorizado")

# Sequencia da aterrissagem com gate
promover_estagio("canario", 2, saudavel=True)
time.sleep(0.2)
promover_estagio("grupo", 25, saudavel=True)
time.sleep(0.2)
promover_estagio("total", 100, saudavel=True)

```

O exemplo mostra a essência operacional do capítulo: cada estágio é uma autorização de pouso separada, com evidência própria — e a arremetida nunca é vergonha, é plano.

O Runbook de Incidente como Artefato de Aterrissagem

O runbook é o procedimento pré-escrito para o momento de pânico — quando o canário acusa anomalia e o tempo conta. O runbook de release agêntico tem etapas fixas:

1. **Confirme a anomalia** com o painel (métrica + janela), nunca pelo e-mail do cliente.
2. **Trave a promoção**: nenhum estágio seguinte recebe tráfego.
3. **Acione o rollback** com o comando pré-testado — a arremetida.
4. **Extraia a caixa-preta**: eventos do agente que produziu a mudança.
5. **Abra o incidente** com evidência anexada, não com narrativa.
6. **Debriefing** no prazo (Capítulo 8) com as lições executáveis.

O runbook transforma o pânico em checklist: no momento de pressão, o time segue o script — e o script foi ensaiado em staging.

A Redundância do Plano de Pouso

Todo release crítico tem plano B, e o plano B tem plano B. O desvio para o cluster standby, o rollback para a versão anterior e o aborto total do release são os três níveis de contingência, cada um testado em staging antes da decolagem. A redundância não é pessimismo — é a constatação de que produção sempre surpreende. O que não pode surpreender é a resposta: com os três níveis treinados, a reação ao imprevisto é execução de procedimento, não improviso sob pressão.

O Fallback como Cidadão de Primeira Classe

O fallback não é um plano B — é um cidadão de primeira classe do ciclo de entrega. Quando o ambiente bloqueia a automação, a esteira degrada com instruções executáveis, nunca em silêncio. Esse padrão — degradar com comandos prontos — é o que mantém a entrega viva em

qualquer ambiente, do sandbox restrito ao pipeline de produção. O Comandante de Operações de Software trata o fallback como parte do design, não como exceção: cada automação nasce com seu par manual documentado.

O Diário do Release

Todo release merece diário — registro cronológico do que foi observado em cada etapa: preparação, canário, expansão gradual, pós-deploy. O diário é o material do pós-mortem e o antídoto da memória seletiva: quando o incidente acontece, o diário mostra o que se sabia e quando se sabia. O diário do release vira também a base do relatório de observabilidade da operação. Sem diário, a análise pós-incidente depende de lembrança; com diário, depende de registro.

Passos para Autorizar o Pouso

1. **Garanta a reprodutibilidade** do build (lockfile + imagem imutável + mesmo commit → mesmo artefato).
2. **Desenhe os estágios** do deploy: canário, grupo progressivo, produção total.
3. **Implemente o gate de saúde** por estágio, com limiares explícitos e rollback automático.
4. **Instrumente o agente** com eventos estruturados de decisão e consumo de contexto.
5. **Escreva o fallback de ambiente** — comandos prontos para execução manual.

Aplica

Cena real, em segunda pessoa. Sua plataforma SaaS promoveu um release agêntico — uma feature de faturamento implementada por agente — direto para produção, porque “o CI estava verde e o prazo apertava”. Duas horas depois, a taxa de erro sobe, o suporte recebe reclamações de cobrança duplicada, e o time descobre que o canário não existia: a mudança foi para 100% dos usuários de uma vez, e o rollback é um processo manual de 40 minutos que ninguém ensaiou.

O erro tem três andares. Primeiro: o release não era reproduzível de forma auditada — “o CI passou na minha máquina” não é evidência de build imutável. Segundo: não houve estágios — o canário foi pulado, e com ele a chance de observar o defeito com 2% de exposição. Terceiro: a observabilidade do agente não existia — ninguém sabia quais decisões o agente tomou na fatura duplicada, porque nenhum evento de decisão foi registrado.

O diagnóstico, ligado à teoria: entrega sem autorização de pouso é um voo sem torre. A correção prática:

1. **Padronize o build imutável** com lockfile e imagem versionada — o mesmo commit sempre produz a mesma imagem.

2. **Imponha os estágios no pipeline:** nada de produção direto; canário, grupo e total com gate de saúde automático.
3. **Ensaie o rollback** antes do release — a arremetida treinada é o plano B que funciona sob pressão.
4. **Instrumente o agente** desde o primeiro dia: evento de decisão por fase, com tokens e evidência.

Armadilhas comuns: tratar o canário como “modo de teste” (canário é observação com tráfego real, não staging); medir apenas uptime (uptime verde com latência degradada é radar cego); e guardar os logs do agente em arquivos que ninguém consulta (observabilidade sem painel é diário pessoal).

Conclusão

Você autorizou o pouso. Três marcos: primeiro, release como artefato reproduzível — build imutável com dependências fixadas; segundo, deploy gradual com gate de saúde e rollback automático — a arremetida como plano B funcional; terceiro, observabilidade em duas camadas — o software e o comportamento do agente, com eventos estruturados de decisão e consumo de contexto.

Como desafio, escreva o playbook de rollback do seu principal serviço e teste-o em staging até ficar automático — a arremetida ensaiada vale mais que a confiança.

No próximo capítulo, você faz o debriefing do voo: o loop de aprendizado que transforma erros de produção em skills, memória e specs revisadas — fechando o ciclo de vida do próprio ciclo.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que autorização de pouso: release e observabilidade. Este capítulo — *Capítulo 7: Autorização de Pouso: Release e Observabilidade* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.
- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.
- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

A rotina de quem já opera assim

Uma semana de trabalho em uma equipe que já adotou o ciclo orientado a agentes não parece uma revolução — parece um fluxo calmo e bem definido. Na segunda-feira, a especificação da semana é revisada em uma reunião curta: cada critério de aceite é lido em voz alta e qualquer ambiguidade é resolvida antes de tocar em código. Na terça, os agentes executam as tarefas em isolamento, enquanto os humanos revisam a arquitetura e os contratos. Na quarta, a verificação roda: testes, revisão adversarial e a decisão de merge apoiada em evidência. Na quinta, o que passou vai para produção em canário, com observabilidade ligada. Na sexta, o debriefing transforma os incidentes da semana em lições e skills. Nenhum dia é heroico; todos os dias são previsíveis. E é exatamente essa previsibilidade — não a velocidade máxima — que define a alta performance.

O que não fazer: os anti-padrões mais comuns

Se você quer destruir o valor do desenvolvimento orientado a agentes, aqui estão as receitas mais eficientes. Primeiro, o prompt-and-pray: gere o código, olhe por cima, e peça desculpas quando quebrar. Funciona em demos, falha em produção. Segundo, a spec decorativa: escreva documentos longos que ninguém verifica e que o agente não consegue executar — o pior dos dois mundos. Terceiro, a auto-verificação: deixe que quem escreveu valide o próprio trabalho, sem revisor independente; é a forma mais rápida de transformar confiança em acidente. Quarto, a delegação sem observabilidade: conceda autonomia sem instrumentar o comportamento. Todos esses padrões têm uma origem comum — a pressa em capturar o ganho sem construir o controle. E todos têm o mesmo antídoto: contrato antes de execução, evidência antes de afirmação, e revisão independente em toda entrega.

Como medir o progresso na prática

Uma dúvida legítima é: como saber se a adoção está dando certo? Métricas tradicionais de velocidade podem enganar — um time pode entregar mais rápido e acumular dívida técnica invisível. O indicador mais confiável no ciclo orientado a agentes é a estabilidade: quantos incidentes em produção, quanto tempo de retrabalho, quantas correções de emergência. Um segundo indicador é o custo de contexto: quantos tokens cada fase consome, e onde o desperdício se concentra. Um terceiro é a taxa de aceite na primeira verificação: se os agentes precisam de muitas rodadas de refutação, o contrato está fraco — o problema não é o agente, é a spec. Com

esses três números na mesa, a conversa de progresso deixa de ser anedótica e vira análise de processo.

O papel do líder neste capítulo

Nada do que este capítulo descreve acontece por acaso — alguém precisa criar as condições para que o processo exista. Esse alguém é o líder técnico, o líder de equipe ou o arquiteto que decidiu tratar o ciclo orientado a agentes como uma mudança de processo, não como a instalação de uma ferramenta. O trabalho do líder aqui tem quatro frentes. A primeira é a modelagem do contrato: garantir que cada fase tem entrada, saída e critério definidos. A segunda é a calibragem da confiança: decidir o que pode ser delegado e o que exige decisão humana, e documentar essa decisão. A terceira é a defesa do tempo de verificação: em uma cultura que celebra velocidade, o líder precisa defender o orçamento de revisão como quem defende o seguro do prédio. A quarta é o exemplo: o líder que pede evidência antes de afirmar, em toda reunião, ensina mais do que qualquer documento de processo.

Perguntas frequentes honestas

P: Isso não vai tirar o emprego dos desenvolvedores? R: A história do ciclo de vida nunca foi sobre menos trabalho humano, mas sobre trabalho mais valioso. O que muda é a natureza da tarefa: escrever código repetitivo deixa de ser o centro, e a especificação, a verificação e o desenho de contratos ocupam o lugar. P: Precisamos de um time de especialistas em IA para começar? R: Não. Precisa-se de disciplina de processo e de vontade de medir. As ferramentas evoluem rápido; o processo é o que permanece. P: E se o agente produzir código que ninguém entende? R: Essa é a pergunta certa — e a resposta é a verificação: se o código passa nos testes, na revisão adversarial e na observabilidade em produção, o fato de ter sido escrito por um agente é irrelevante. O critério não é a origem, é a evidência. P: Quanto tempo leva para ver resultado? R: Na primeira semana você já vê o efeito de escrever critérios de aceite verificáveis, independentemente de agentes. Os ganhos estruturais aparecem em um a dois ciclos.

Um convite para a prática deliberada

Conhecimento sem prática é entretenimento disfarçado de aprendizado. Este capítulo termina com um convite para a prática deliberada: escolha um artefato real do seu trabalho — uma spec, um teste, um release — e aplique deliberadamente um dos conceitos aqui descritos. Anote o

antes e o depois. Repita por quatro semanas. No fim do mês, compare: o processo está mais previsível? O custo de contexto caiu? A estabilidade melhorou? Esse experimento pessoal, pequeno e mensurável, vale mais do que qualquer curso. É assim que o ciclo orientado a agentes deixa de ser um conceito que você explica para outras pessoas e se torna uma capacidade que você demonstra.

Síntese para levar com você

Se você guardar apenas uma ideia deste capítulo, que seja esta: no ciclo orientado a agentes, o contrato precede a execução, a evidência precede a afirmação e a revisão independente precede a entrega. Tudo o mais — as ferramentas, os modelos, os fluxos — muda rápido e pode ser aprendido conforme a necessidade. O que não muda é a disciplina: sem ela, a IA é um gerador de volume; com ela, é um multiplicador de capacidade. O resto do livro é a expansão dessa disciplina em cada fase do ciclo de vida.

De onde veio a necessidade desta mudança

Vale a pena entender por que este capítulo existe — e por que ele não foi escrito dez anos atrás. A resposta está na economia do desenvolvimento de software. Durante décadas, o custo dominante de produzir software foi o trabalho humano: escrever, revisar, corrigir. Todo o ciclo de vida clássico foi desenhado em torno dessa escassez — processos, papéis e artefatos existem para coordenar pessoas e evitar retrabalho caro. O que mudou nos últimos anos foi a emergência de modelos capazes de gerar, revisar e executar código com custo marginal próximo de zero. De repente, a escassez dominante não é mais a mão de obra: é a capacidade de especificar, orquestrar e verificar. Esse deslocamento — de horas-homem para tokens e contexto — é a raiz de tudo o que este capítulo descreve. Quem entende essa mudança de economia entende por que o processo precisa mudar junto com a ferramenta.

Conversando com quem resiste

Em toda equipe há quem resista à mudança — e a resistência quase nunca é preguiça, é uma pergunta legítima sem resposta. As objeções mais comuns são três. “Já tentamos automação e quebrou”: a resposta é que a automação anterior quebrou porque o processo não tinha contrato nem verificação; é exatamente isso que o novo ciclo constrói antes de automatizar. “IA gera código que ninguém entende”: a resposta é que o critério de entendimento mudou — o que

importa não é a origem do código, mas se ele passa na verificação; e a revisão de contrato e arquitetura continua humana. “Isso é modismo”: a resposta mais honesta é que pode ser, mas o processo que este capítulo descreve — especificar, delegar, verificar, aprender — melhora o ciclo com ou sem IA. A disciplina é o investimento à prova de modismo.

O dia a dia no detalhe

Para tornar concreto o que este capítulo descreve, vale percorrer o dia a dia de uma tarefa típica, passo a passo. A manhã começa com a revisão da intenção: o produto explica o que quer, o time traduz em requisitos com critérios verificáveis e ninguém toca em código antes de a spec estar aprovada. Na sequência, o trabalho é despachado: cada tarefa vai para um contexto isolado, com seu contrato anexado. A execução produz artefatos — código, testes, diagramas — e cada artefato carrega a evidência de como foi produzido. A tarde é de verificação: testes automáticos, revisão adversarial e a leitura humana do que é crítico. O que passa, segue; o que não passa, volta com o parecer anexado — sem discussão de opinião, porque o critério já estava escrito. O fim do dia é de registro: o que foi aprendido, o que custou em contexto, o que deve mudar no processo. Esse fluxo parece simples, mas cada passo exige disciplina — e é exatamente a simplicidade do ritmo que o torna sustentável.

O custo invisível que decide tudo

Há um recurso que atravessa todos os exemplos deste capítulo e que raramente aparece nas discussões: o contexto. Cada interação com um modelo de linguagem consome uma janela de informação — e essa janela é limitada e cara. Uma spec mal escrita gasta contexto em ciclos de correção. Um log inteiro no contexto gasta contexto que poderia servir à verificação. Uma busca redundante gasta contexto sem produzir informação. Quem ignora esse custo descobre, cedo ou tarde, que a automação ficou mais cara que o trabalho manual que pretendia substituir. Por isso a disciplina de contexto não é um detalhe de economia — é uma decisão de arquitetura do ciclo. Medir o consumo por fase, comprimir o que é ruído e injetar apenas o necessário são práticas que determinam se o SDLC AI-first se sustenta em escala. Este capítulo toca nesse tema; os capítulos finais do livro o desdobram em técnica.

Uma visão de longo prazo

A adoção do ciclo orientado a agentes não é um projeto com data de fim — é uma trajetória que se desenrola ao longo de anos, e vale a pena olhar para a frente. No primeiro trimestre, o foco é o contrato: as equipes aprendem a especificar e a verificar, e os ganhos vêm da clareza, não da automação. No segundo trimestre, a delegação supervisionada entra em produção em áreas de baixo risco, e as métricas começam a mostrar onde o ciclo ganha e onde perde. No segundo ano, o ciclo adversarial se consolida: verificação automática, observabilidade do comportamento agêntico e aprendizado organizacional rodando como rotina. No terceiro ano, a organização opera em um nível de maturidade em que a IA é parte estrutural do processo, e a pergunta deixa de ser “como adotar” e passa a ser “como evoluir”. Quem inicia com o processo antes da ferramenta chega a esse destino com estabilidade; quem inverte a ordem, chega com dívida. A escolha é sua.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

